

Automation

Prof. Gerardo Flores

February, 2026

Robotics and Automation course,
TAMIU



MOTIVATION: Proportional Control Example

We consider a simple first-order system:

$$\dot{x} = u$$

To make the state $x(t)$ converge to a desired value x_d , we apply a **proportional control law**:

$$u = -k(x - x_d)$$

Substituting into the system dynamics gives:

$$\dot{x} = -k(x - x_d)$$

This results in **exponential convergence** of the error $e = x - x_d$ toward zero. The system is stable for any gain $k > 0$.

Proportional Control – 1st Order System Example

This example simulates a simple first-order dynamic system:

$$\dot{x} = u$$

A proportional controller is applied to drive the state $x(t)$ toward a desired target x_d :

$$u = -k(x - x_d)$$

This results in an exponential decay of the tracking error $e = x - x_d$.

The system is integrated using Euler's method, and the evolution of $x(t)$ over time is plotted.

The plot shows how the system smoothly converges to the desired value.

Problem: Simulating a First-Order System with Proportional Control

Objective:

Simulate and visualize a first-order system controlled by a proportional controller, where the state $x(t)$ converges to a desired reference value x_d .

System Dynamics:

We consider the continuous-time first-order system:

$$\dot{x}(t) = u(t)$$

The control input is defined as:

$$u(t) = -k(x(t) - x_d)$$

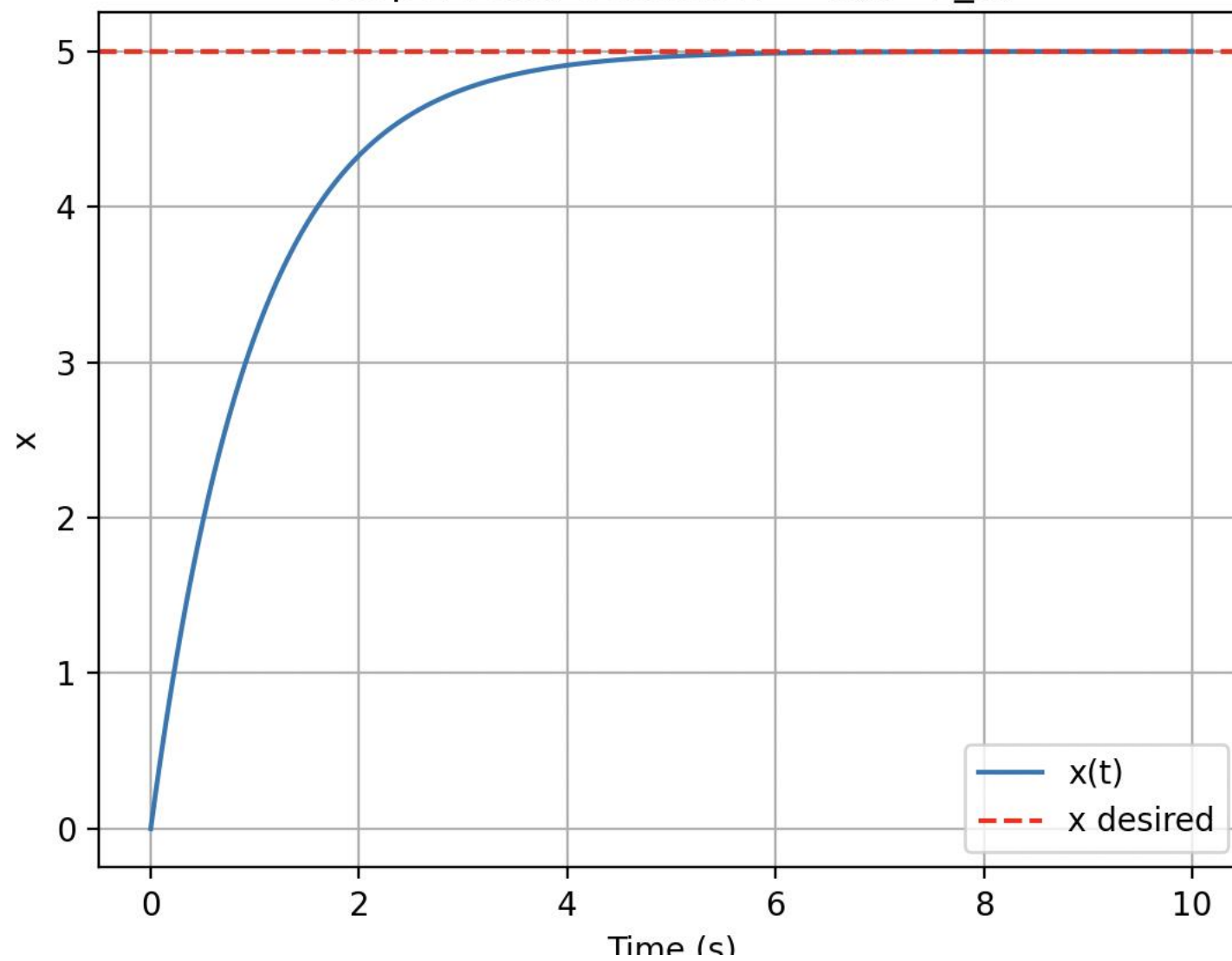
This results in the **closed-loop system**:

$$\dot{x}(t) = -k(x(t) - x_d)$$

Simulation Approach:

We solve this system numerically using **Euler integration** in Python.

Proportional Control: $\dot{x} = -k(x - x_d)$



Step-by-Step Solution of the Closed-Loop Differential Equation

1. Original system:

$$\dot{x} = u$$

2. Proportional control law:

$$u = -k(x - x_d)$$

Substitute into the system:

$$\dot{x} = -k(x - x_d)$$

3. Express in terms of error:

Define the tracking error:

$$e(t) = x(t) - x_d$$

Since x_d is constant:

$$\dot{e} = \dot{x}$$

So the system becomes:

$$\dot{e} = -ke$$

4. Solve the differential equation: $\dot{e}(t) = -ke(t)$

1. Start with the differential equation:

$$\frac{de}{dt} = -ke$$

2. Separate the variables:

$$\frac{1}{e} de = -k dt$$

3. Integrate both sides:

$$\int \frac{1}{e} de = \int -k dt$$
$$\ln e = -kt + C$$

4. Exponentiate both sides:

$$e(t) = e^{-kt+C} = e^C \cdot e^{-kt}$$

Let $A = e^C$, so:

$$e(t) = Ae^{-kt}$$

5. Apply the initial condition $e(0) = e_0$:

$$e(0) = Ae^0 = A \Rightarrow A = e_0$$

6. Final solution:

$$e(t) = e_0 \cdot e^{-kt}$$

And since $e(t) = x(t) - x_d$, we can express:

$$x(t) = x_d + (x_0 - x_d)e^{-kt}$$

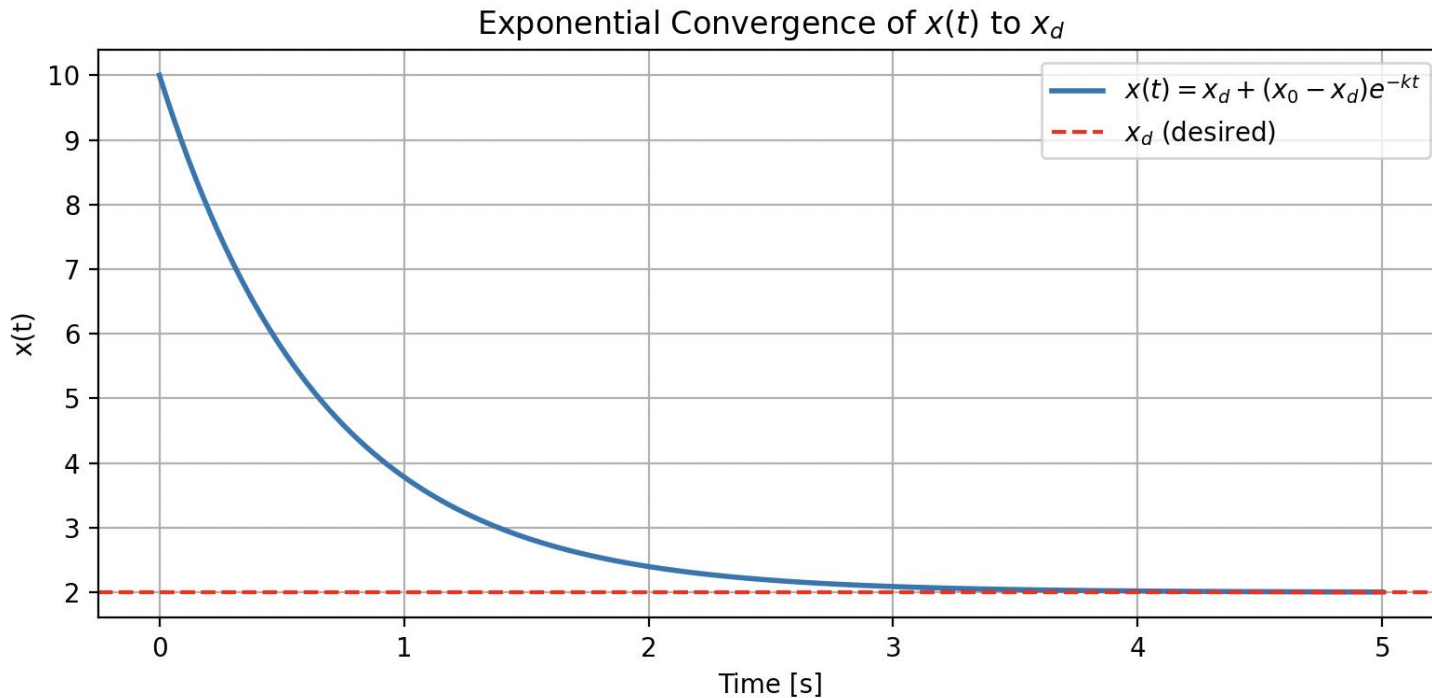
Example: Exponential Convergence of a First-Order System

We want to visualize the solution to the equation:

$$x(t) = x_d + (x_0 - x_d)e^{-kt}$$

This represents the evolution of a state $x(t)$ converging exponentially toward the desired value x_d , starting from an initial condition x_0 . The gain k controls the speed of convergence.

Below is a simple Python implementation:



This plot shows how $x(t)$ starts at $x_0 = 10$, and smoothly converges to $x_d = 2$ with a rate defined by $k = 1.5$.

4. Solve the differential equation:

This is a first-order linear differential equation. The general solution is:

$$e(t) = e(0)e^{-kt}$$

Where $e(0) = x(0) - x_d$

5. Recover $x(t)$:

$$x(t) = e(t) + x_d = (x_0 - x_d)e^{-kt} + x_d$$

Final solution:

$$x(t) = x_d + (x_0 - x_d)e^{-kt}$$

Interpretation:

- The state $x(t)$ converges exponentially to the desired value x_d .
- The gain $k > 0$ determines how fast the system settles.
- A larger k results in faster convergence.

Recap of structured control procedure:

1. Identify the system:

$$\dot{x} = u \quad (\text{Control input: } u \in \mathbb{R})$$

2. Define the error:

$$e = x - x_d \quad (\text{desired state: } x_d)$$

3. Design the control law:

$$u = -k(x - x_d)$$

4. Analyze the closed-loop behavior:

$$\dot{x} = -k(x - x_d) \quad \Rightarrow \quad x(t) \rightarrow x_d \text{ as } t \rightarrow \infty$$

5. Simulate the system:

Use numerical integration (e.g., Euler or `solve_ivp`) to observe the response.

6. Implement the control:

Apply the control law in a real system or embedded controller using the error feedback.

The rigid body equations

$$\left. \begin{aligned} \dot{p} &= \omega \times p + v \\ \dot{R}(t) &= S(\omega(t)) R(t) \end{aligned} \right\} \text{Kynematics}$$

$$\left. \begin{aligned} m \dot{v} &= F_{\text{ext}} \\ I \dot{\omega} + \omega \times (I \omega) &= \tau_{\text{ext}} \end{aligned} \right\} \text{Dynamics}$$

write them in scalar form for the drone.

Note: This simplified form is used when the body's reference origin is chosen as the rotation point, i.e., $o = 0$.

Scalar form (body frame aligned with principal axes) $I \dot{\omega} + \omega \times (I \omega) = \tau_{\text{ext}}$

Assume

$$\boldsymbol{\omega} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \equiv \begin{bmatrix} \text{roll rate} \\ \text{pitch rate} \\ \text{yaw rate} \end{bmatrix}, \quad \mathbf{I} = \text{diag}(I_x, I_y, I_z), \quad \boldsymbol{\tau}_{\text{ext}} = \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix}.$$

Then the rotational dynamics become the three coupled scalar equations

$I_x \dot{p} + (I_z - I_y) qr = \tau_\phi$	(roll)
$I_y \dot{q} + (I_x - I_z) pr = \tau_\theta$	(pitch)
$I_z \dot{r} + (I_y - I_x) pq = \tau_\psi$	(yaw)

For the translation $m \dot{v} = F_{\text{ext}}$ we only care about the vertical (altitude) direction:

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}, \quad \mathbf{F}_{\text{ext}} = \begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix}$$

and we keep just the z -component

$$m\dot{v}_z = F_z = T_{\text{total}} - mg + F_{z,\text{dist}}$$

where T_{total} is the total thrust from the four rotors (is the control input), mg is the weight, and $F_{z,\text{dist}}$ represents vertical disturbances.



1-control-UAV-altitude.py